

Aula 7



DIAGRAMA ER

Um diagrama de relacionamento de entidade (diagrama ER ou ERD) é uma representação visual de como os itens em um banco de dados se relacionam entre si.

Os ERDs são um tipo especializado de fluxograma que transmite os tipos de relacionamentos entre diferentes entidades dentro de um sistema.

Eles usam um conjunto definido de símbolos, e os vinculam a linhas de conexão.:

- incluindo retângulos,

- ovais

- losangos

PARA QUE SÃO USADOS OS ERDS?

- Resolução de problemas de bancos de dados
- Reengenharia de processos de negócios (BPR)
- Projeto de bancos de dados e modelagem de dados

Como ferramentas de modelagem de dados para avaliar o escopo dos bancos de dados que suas organizações precisam, e então planejam como os dados serão armazenados.

O ERD representa o nível conceitual.

TIPO 'CHEN' (CLASSICA)

Características:

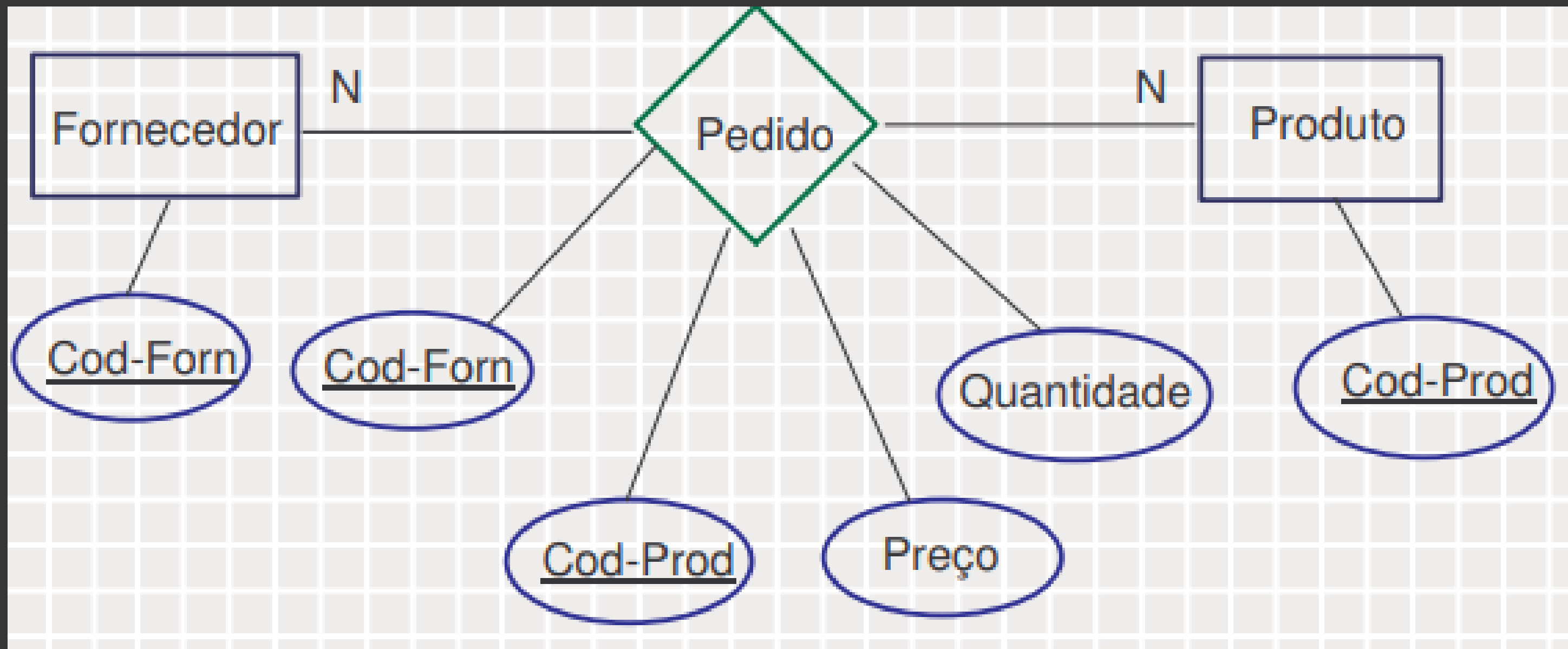
- Entidades = retângulos
- Atributos = elipses (ovais)
- Relacionamentos = losangos
- Chave primária = atributo sublinhado



Uso:

- Boa para ensino/aprendizado (mais didática, mostra atributos explicitamente).
- Mais usada em fase conceitual, quando estamos explicando a lógica do domínio para alguém que não conhece SQL.

TIPO 'CHEN' (CLASSICA)



TIPO ‘PÉ DE GALINHA’

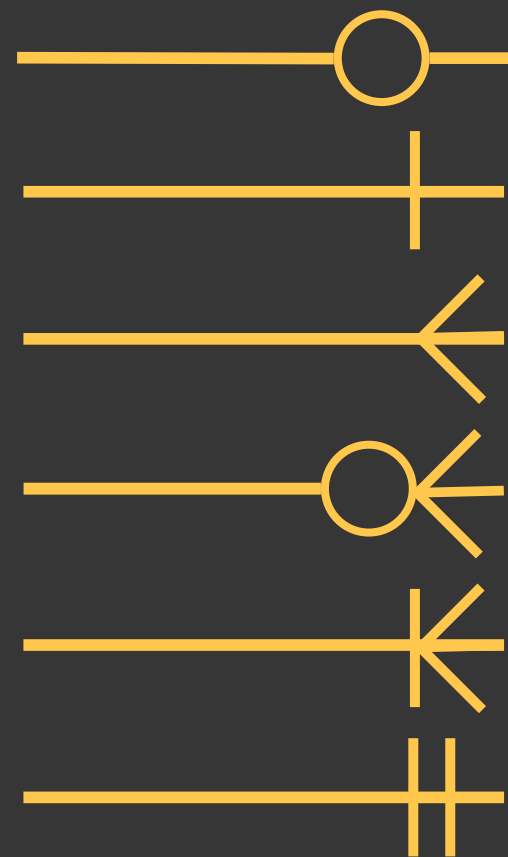
Características:

- Entidades = tabelas (retângulos com colunas dentro).
- Relacionamentos = linhas com símbolos (pé de galinha para N, barra para 1, círculo para opcionalidade).
- Não mostra atributos como elipses separados, já ficam listados dentro da entidade.

Uso:

- Muito usada em fase lógica (quando você já pensa em tabelas e relacionamentos).
- Mais prática para projetos reais porque se parece com o modelo relacional.
- Ferramentas como MySQL Workbench, SQL Server Management Studio, Oracle Data Modeler usam essa notação.

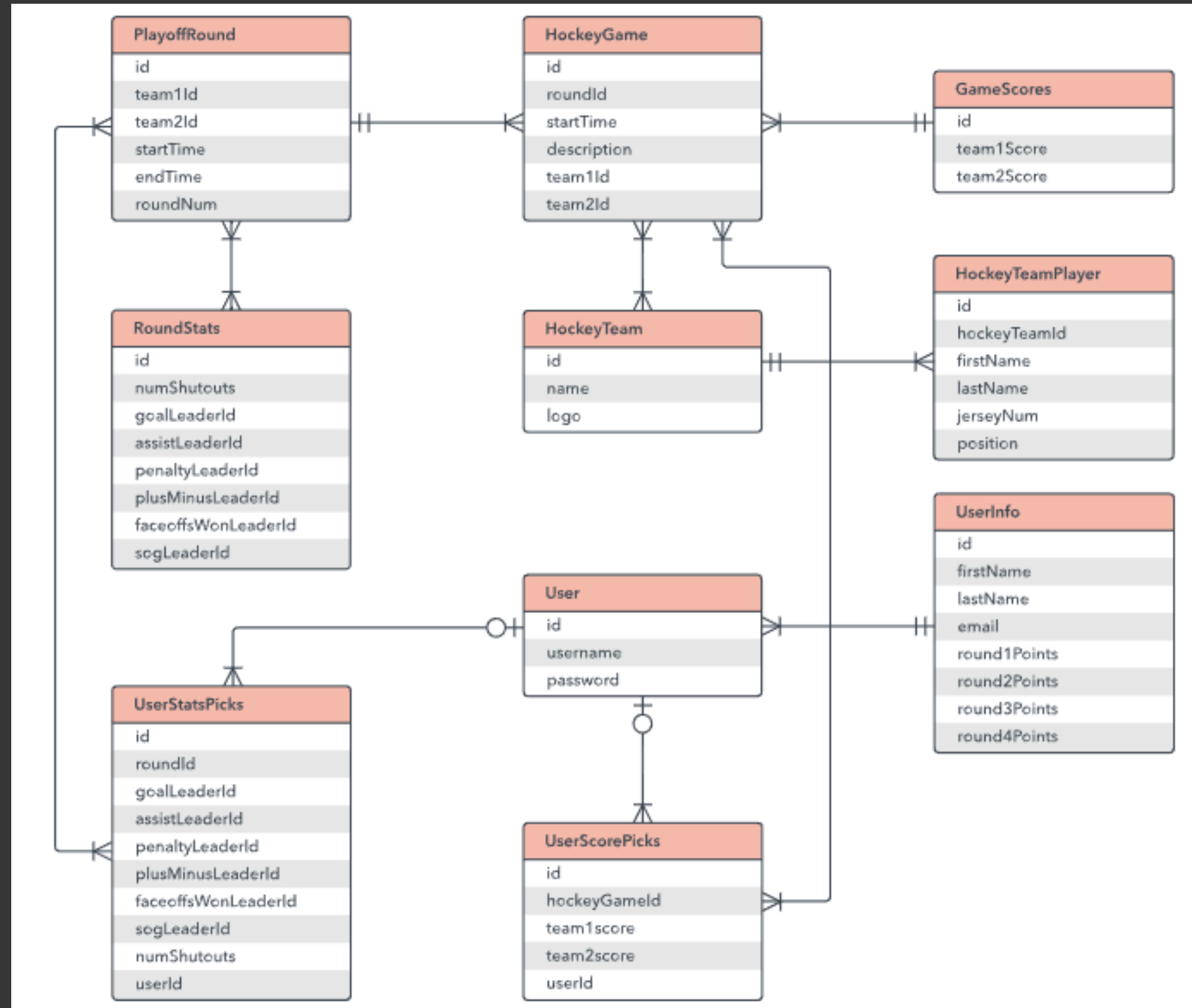
TIPO ‘PÉ DE GALINHA’



Características:

- ZERO
- UM
- MUITOS
- ZERO OU MUITOS
- UM OU MUITOS
- UM E APENAS UM

TIPO 'PÉ DE GALINHA'



QUANDO USAR

CHEN

FASE CONCEITUAL

PÉ DE GALINHA

FASE LÓGICA

CONSULTAS - SELECT

Selecionar algumas colunas da tabela

```
SELECT nome , email FROM Aluno ;
```

Selecionar todas as colunas da tabela

```
SELECT * FROM Aluno ;
```

Selecionar todas as colunas de mais de uma tabela

```
SELECT * FROM Aluno, Professor ;
```

Selecionar colunas com o mesmo nome em tabelas diferentes

```
SELECT Aluno . nome , Professor . nome FROM Aluno , Professor ;
```

Selecionar e utilizar apelidos para as colunas

```
SELECT a . nome AS aluno_nome , p . nome AS professor_nome FROM Aluno AS a , Professor AS p;
```

CONSULTAS - WHERE

Selecionar com filtro

```
SELECT * FROM Aluno WHERE altura > 1.80;
```

```
SELECT * FROM Aluno WHERE altura = 1.8;
```

```
SELECT * FROM Aluno WHERE altura <> 1.8;
```

```
SELECT * FROM Aluno WHERE altura != 1.8;
```

```
SELECT * FROM Aluno WHERE altura <= 1.8;
```

```
SELECT * FROM Aluno WHERE altura < 1.8;
```

```
SELECT * FROM Aluno WHERE altura >= 1.8;
```

```
SELECT * FROM Aluno WHERE altura > 1.8;
```

CONSULTAS - WHERE

Selecionar com filtro

IS TRUE, IS NOT TRUE, IS NULL, IS NOT NULL, BETWEEN, IS NOT BETWEEN

```
SELECT * FROM Aluno WHERE aprovado IS TRUE ;
```

```
SELECT * FROM Aluno WHERE aprovado IS NOT TRUE ;
```

```
SELECT * FROM Aluno WHERE nome IS NULL ;
```

```
SELECT * FROM Aluno WHERE nome IS NOT NULL ;
```

```
SELECT * FROM Aluno WHERE altura BETWEEN 1.5 AND 1.8;
```

```
SELECT * FROM Aluno WHERE altura NOT BETWEEN 1.5 AND 1.8;
```

CONSULTAS - WHERE

Selecionar com filtro

LIKE, NOT LIKE, IN, NOT IN

```
SELECT * FROM Aluno WHERE nome LIKE 'Rafael %';
```

```
SELECT * FROM Aluno WHERE nome NOT LIKE 'Rafael %';SELECT *
```

```
SELECT * FROM Aluno WHERE altura IN (1.5 , 1.6 , 1.7 , 1.8) ;
```

```
SELECT * FROM Aluno WHERE altura NOT IN (1.5 , 1.6 , 1.7 , 1.8) ;
```

CONSULTAS - WHERE

Selecionar com filtro

LIKE, NOT LIKE, IN, NOT IN

```
SELECT * FROM Aluno WHERE nome LIKE 'Rafael %';
```

```
SELECT * FROM Aluno WHERE nome NOT LIKE 'Rafael %';SELECT *
```

```
SELECT * FROM Aluno WHERE altura IN (1.5 , 1.6 , 1.7 , 1.8) ;
```

```
SELECT * FROM Aluno WHERE altura NOT IN (1.5 , 1.6 , 1.7 , 1.8) ;
```

CONSULTAS - ORDER BY

Ordenar com

ORDER BY, DESC ou ASC

```
SELECT * FROM Aluno ORDER BY altura ;
```

```
SELECT * FROM Aluno ORDER BY altura , nome ;
```

```
SELECT * FROM Aluno ORDER BY altura DESC , nome ASC ;
```

CONSULTAS - FUNÇÕES

Realizar funções com

COUNT, AVG, SUM, MAX, MIN, VARIANCE

```
SELECT COUNT (*) FROM Aluno ; (para Contagem)
```

```
SELECT AVG( altura ) FROM Aluno ; (para Obter Média Aritmética)
```

```
SELECT SUM( altura ) FROM Aluno ; (para Obter Soma)
```

```
SELECT MAX( altura ) FROM Aluno ; (para Obter o valor máximo)
```

```
SELECT MIN( altura ) FROM Aluno ; (para Obter o valor mínimo)
```

```
SELECT VARIANCE ( altura ) FROM Aluno ; (para calculo de amplitude )
```

CONSULTAS - GROUP BY

Agrupar com

GROUP BY

```
SELECT aprovado , COUNT (*) FROM Aluno GROUP BY aprovado ;
```

```
SELECT sexo , aprovado , COUNT (*) FROM Aluno GROUP BY sexo , aprovado ;
```

CONSULTAS - DISTINCT

Eliminar itens repetidos com

DISTINCT

```
SELECT DISTINCT ( cidade ) FROM Aluno ;
```

CONSULTAS - LIMIT

Limitar a nossa consulta com

LIMIT

```
SELECT * FROM Aluno LIMIT 10;
```

```
SELECT * FROM Aluno LIMIT 5 , 10; (descarta as 5 primeiras)
```

SUBQUERY

Uma **subquery** é uma query como qualquer outra, porém ela é executada dentro de uma outra query de SELECT, INSERT, UPDATE ou DELETE. A função da subquery é produzir um resultado que será utilizado pela query que a contém. Alguns autores se referem à subquery como query interna e a query que a contém como query externa.

```
SELECT *  
FROM Nota AS n1  
WHERE n1 . nota > (  
  SELECT AVG( n2 . nota )  
  FROM Nota AS n2  
  WHERE n2 . turma_id = n1 . turma_id  
);
```

JOINS

Utilizamos **JOIN** do SQL para extrairmos as informações de uma ou mais tabelas em um único conjunto de resultados baseando-se nos relacionamentos entre as colunas das tabelas envolvidas.

Para trazer as informações das duas tabelas em um único conjunto de resultados utilizaremos a instrução JOIN.

```
SELECT *  
FROM Usuario AS u  
JOIN Perfil AS p
```

JOINS

Para definirmos qual a coluna irá definir o relacionamento entre as tabelas devemos utilizar a instrução **JOIN** juntamente com a instrução **ON**.

```
SELECT *  
FROM Usuario AS u  
JOIN Perfil AS p  
ON u . id = p . id ;
```

JOINS

JOIN: retorna registros quando existe algum valor na coluna de relacionamento em pelo menos uma das tabelas.

LEFT JOIN: retorna registros quando existe algum valor na coluna de relacionamento da tabela informada à esquerda na consulta.

RIGHT JOIN: retorna registros quando existe algum valor na coluna de relacionamento da tabela informada à direita na consulta.

FULL JOIN: retorna todos os registros de ambas as tabelas, combinando os que têm correspondência na condição ON. Quando não há correspondência, os campos da tabela que não tem relação ficam com valor NULL.

```
SELECT *  
FROM Usuario AS u  
LEFT JOIN Perfil AS p  
ON u.id = p.id;
```

```
SELECT *  
FROM Usuario AS u  
RIGHT JOIN Perfil AS p  
ON u.id = p.id;
```

```
SELECT *  
FROM Usuario AS u  
FULL OUTER JOIN Perfil AS p  
ON u.id = p.id;
```

UNION

Combina os resultados de duas ou mais consultas SELECT em um único conjunto de resultados, automaticamente eliminando linhas duplicadas. Para usá-lo, todas as consultas devem ter o mesmo número de colunas e tipos de dados compatíveis nas colunas correspondentes.

```
SELECT nome_usuario , email  
FROM Usuario  
  
UNION  
  
SELECT nome_usuario , email  
FROM UsuarioAdministrativo ;
```

UNION ALL

Se quisermos que o resultado traga entradas duplicadas, devemos utilizar a instrução UNION ALL.

```
SELECT nome_usuario , email
```

```
FROM Usuario
```

```
UNION ALL
```

```
SELECT nome_usuario , email
```

```
FROM UsuarioAdministrativo ;
```